Matplotlib
Introduction

`Matplotlib` is the name of a famous package in python, for plotting different graphs and visualize behavior of a system with animations.

You can Install it via different package managers, including `pip` . And the command is: `pip install matplotlib` . Feel free to use any other method of installation, visiting ...
https://matplotlib.org/stable/index.html

TABLE OF CONTENTS

- How to Plot a Graph?
- Different Genre of Plots
- Sub Plots
- Simple Sine Animation
- Exercises

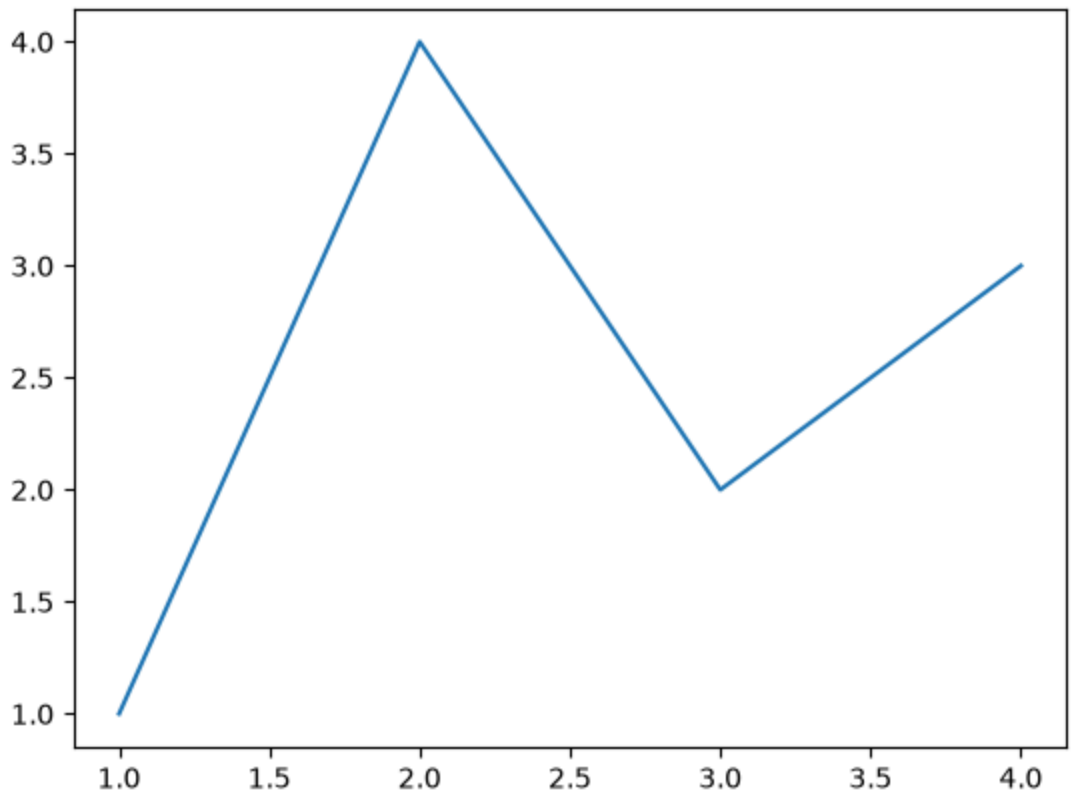
How to Plot a Graph?

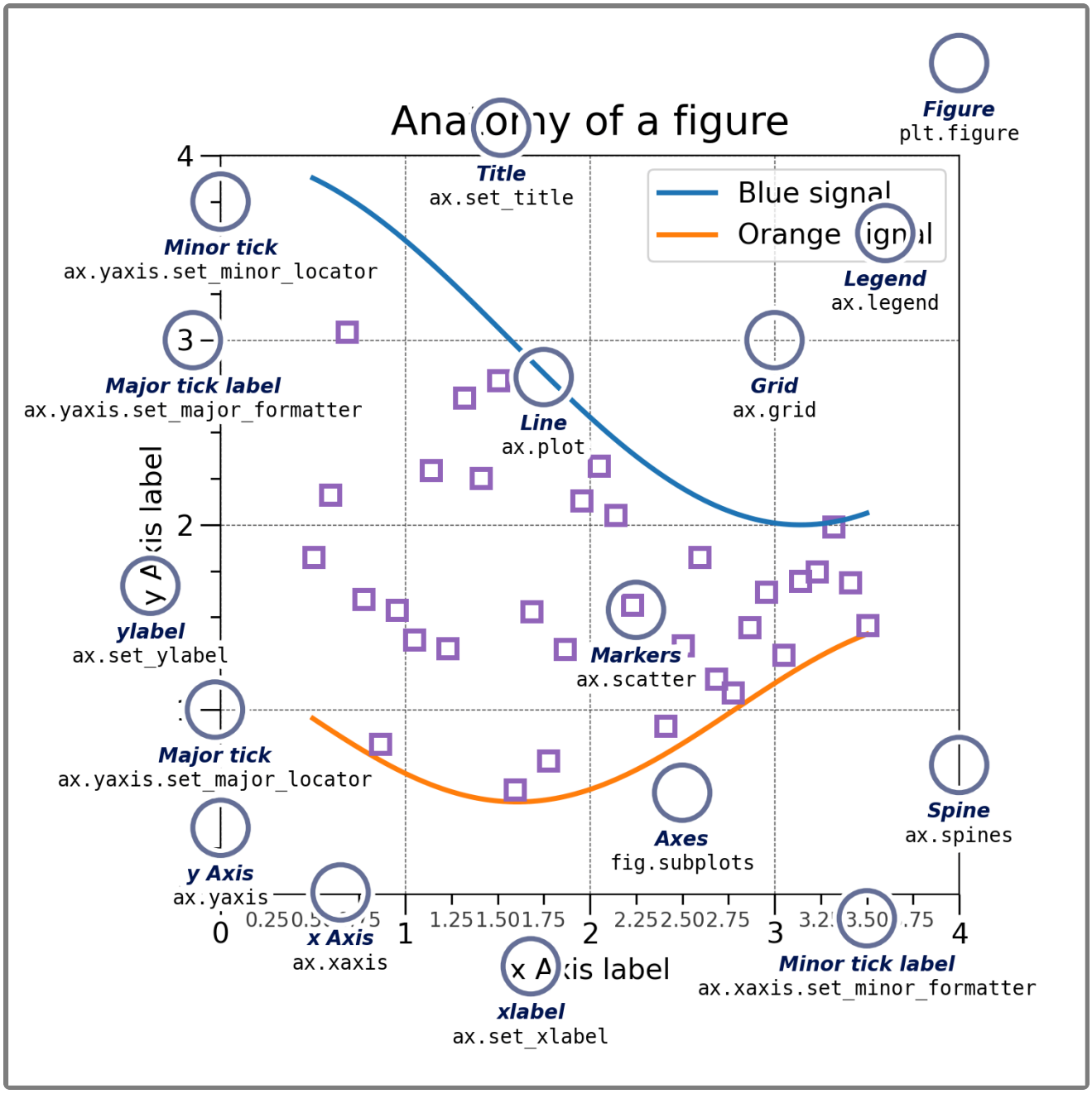
In general, for graphing a plot, we need a set of `x` axis values, and a set of `y` axis values. The easiest way to draw a graph is by passing these lists of `x` and `y` values and `matplotlib` will connect those dots together.

```
import matplotlib.pyplot as plt

xs = [1,2,3,4]
ys = [1,4,2,3]

plt.plot(xs, ys)
```





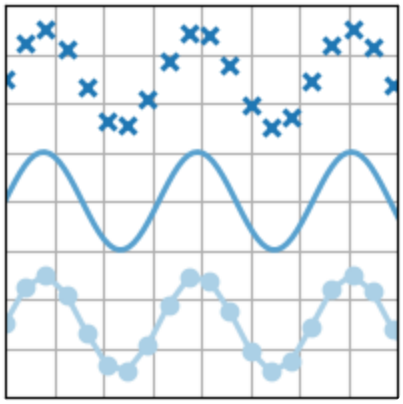
Simple `plot` function arguments

- `xs`
- `ys`
- **`color` or `c`** : Line color.
- **`linestyle` or `ls`** : Line style (e.g., `'-'`, `'--'`, `':'`).
- **`linewidth` or `lw`** : Line width in points.
- **`marker`** : Marker style (e.g., `'o'`, `'^'`, `'s'`).
- **`markersize` or `ms`** : Marker size.
- `alpha` : Setting the opacity of the graph e.g. `alpha=0.5`

Different Genre of Plots

You can see a list of popular plots, frequently drawn in physical projects, with their code. Visit https://matplotlib.org/stable/plot_types/index.html for more samples.

PLOT(X,Y)

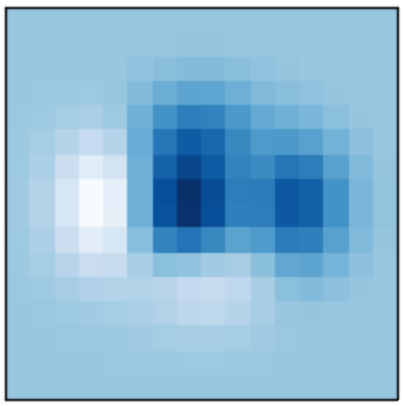


```
fig, ax = plt.subplots(
    nrows=1,
    ncols=2,
    sharey=True,
    figsize = (8,4),    # dimensions of all plots together.
    # width_ratios=[4, 1], # height_ratios=[] for column
    layout='constrained' # Prevents overlapping titles/labels automatically
)
ax[0].plot(xs, ys, color="purple", linewidth=5)
ax[0].plot(xs, ys_n, color="red", linewidth=1, linestyle="dashed")
ax[0].set_title("Unnormalized")
ax[0].grid()
ax[0].legend(labels = ["Unnormalized", "Normalized"], loc='upper right')
ax[0].set_ylabel("y")
ax[0].set_xlabel("x")
ax[0].yaxis.set_minor_locator(MultipleLocator(0.05))

ax[1].plot(xs, ys_n, color="purple", linewidth=1)
ax[1].set_title("Normalized")
ax[1].set_xlabel("x")
ax[1].grid()

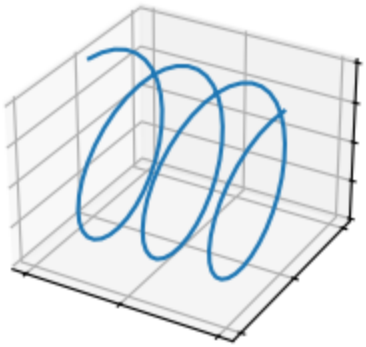
plt.show()
```

IMSHOW(Z)



Parameter	Core Function	Common Values and Considerations
`X`	Input data	2D array (grayscale/values) or 3D array (RGI
`cmap`	Colormap	`'gray'` (grayscale), `'hot'` (heatmap), `'vi` etc.
`origin`	Image orientation	`'upper'` (default, `[0,0]` at top-left) or `'l` bottom-left) .
`extent`	Coordinate range	`[left, right, bottom, top]`, used to rep with actual physical coordinates .
`interpolation`	Interpolation method	`'nearest'` (sharp, suitable for data), `'bil` suitable for photos) .
`vmin` / `vmax`	Color scale range	Manually specify the minimum and maximu mapping, often used to fix the color scale acr images for comparison .
`alpha`	Transparency	0 (fully transparent) to 1 (fully opaque), used other images or contour lines .
`aspect`	Aspect ratio	`'auto'` (stretch to fill), `'equal'` (1:1 pixel is usually `'equal'` .

PLOT(XS, YS, ZS)



```
fig = plt.figure()

ax = fig.add_subplot(projection='3d')

ax.plot(xs, ys, zs, linewidth=1.5, color='red')

ax.set_box_aspect((10, 10, 10))
```

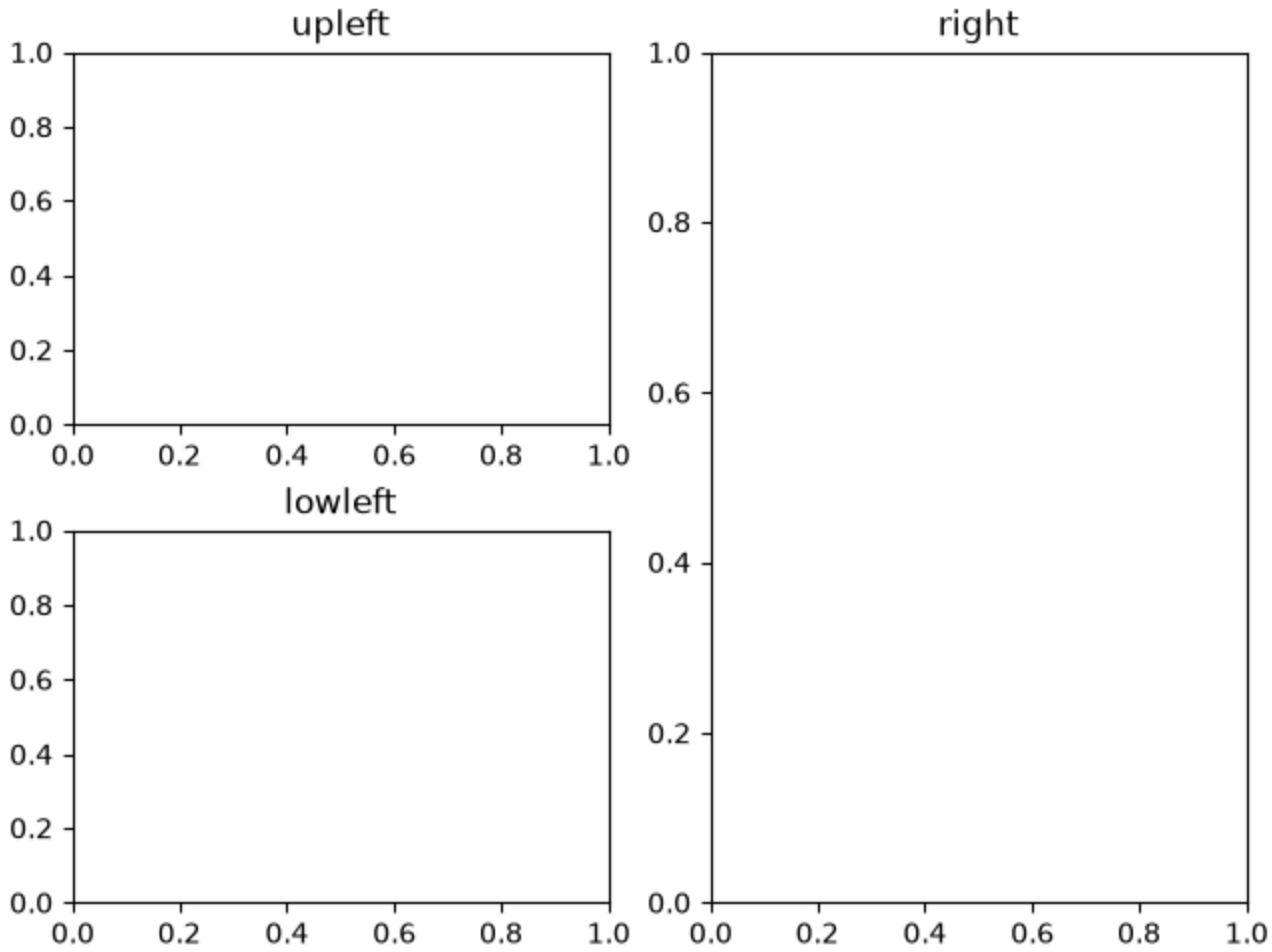
```
# ax.view_init(elev=-50, azim=30, roll = 90)
ax.set_xlabel('x-axis [m]')
ax.set_ylabel('y-axis [m]')
ax.set_zlabel('z-axis [m]')
ax.set_title('Electron Helix in B-field')
ax.xaxis.pane.fill = True
ax.grid(True)

plt.show()
```

Subplots

In order to compare a few graphs and figures next to each other we can use subplots and arrange each individual plot next to the others.

```
fig, axd = plt.subplot_mosaic([['upleft', 'right'],
                               ['lowleft', 'right']], layout='constrained')
axd['upleft'].set_title('upleft')
axd['lowleft'].set_title('lowleft')
axd['right'].set_title('right')
```



Simple Sine Animation

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

```
# 1. Setup figure and initial artist
fig, ax = plt.subplots()
x = np.linspace(0, 2 * np.pi, 200)
y = np.sin(x)
line, = ax.plot([], [], lw=2) # Start with empty data
ax.set_xlim(0, 2 * np.pi)
ax.set_ylim(-1.2, 1.2)

# 2. Define update function
def update(frame):
    # frame is the current index from the generator
    line.set_data(x[:frame], y[:frame])
    return line,

# 3. Create the animation
# frames is a generator yielding 0, 1, 2... up to len(x)
ani = animation.FuncAnimation(
    fig,
    update,
    frames=len(x),
    interval=20,
    blit=True # blit=True improves performance by redrawing only changed parts
)

plt.show()
```

LINE-BY-LINE EXPLANATION OF THE ANIMATION CODE

Here is the complete code again for reference, followed by a detailed breakdown of every line.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
fig, ax = plt.subplots()
x = np.linspace(0, 2 * np.pi, 200)
y = np.sin(x)
line, = ax.plot([], [], lw=2)
ax.set_xlim(0, 2 * np.pi)
ax.set_ylim(-1.2, 1.2)
def update(frame):
    line.set_data(x[:frame], y[:frame])
    return line,
ani = animation.FuncAnimation(
    fig, update, frames=len(x), interval=20, blit=True
)
plt.show()
```

IMPORTS

```
import numpy as np
```

Imports NumPy under the alias `np`. NumPy provides fast numerical arrays and math functions (like `linspace` and `sin`) that we need to generate the data to animate.

```
import matplotlib.pyplot as plt

Imports Matplotlib's pyplot interface under `plt`. This gives you the familiar plotting
commands (`subplots`, `plot`, `show`) and manages the figure/window lifecycle.
```

```
import matplotlib.animation as animation
```

Imports the animation submodule. This is where `FuncAnimation` lives. It is **not** automatically loaded by `pyplot`, so you must import it explicitly to create animations.

FIGURE AND AXES SETUP

```
fig, ax = plt.subplots()
```

Creates a new figure and a single set of axes.

- `fig` is the top-level container (the window / canvas).
- `ax` is the drawing area where the plot is rendered.
  `FuncAnimation` needs the `fig` object, and we need `ax` to configure the axes and draw the line.

GENERATING THE DATA

```
x = np.linspace(0, 2 * np.pi, 200)
```

Creates an array of **200 evenly spaced numbers** between `0` and `2π` (≈ 6.283). This is the x-domain for a single full sine cycle.

```
y = np.sin(x)
```

Applies sine element-wise to `x`, producing an array of 200 y-values. Together, `(x, y)` traces one full sine wave.

CREATING THE INITIAL (EMPTY) LINE ARTIST

```
line, = ax.plot([], [], lw=2)
```

This is the most important line to understand for animations.

- `ax.plot([], [])` creates a Line2D artist with **empty data**, so nothing is visible yet.
- `lw=2` sets the line width to 2.
- `ax.plot(...)` returns a **list** of line objects (one per line plotted). The trailing comma in `line, = ...` **unpacks** that list so `line` is the single `Line2D` object itself, not a list.

Why unpack? Because later we call `line.set_data(...)` — that method exists on the `Line2D` object, not on the list. If you wrote `line = ax.plot(...)`, you would have a list and `line.set_data` would fail.

The key insight: **we create the artist once and mutate its data every frame**, rather than re-plotting each frame.

SETTING THE AXES LIMITS

```
ax.set_xlim(0, 2 * np.pi)
```

Fixes the x-axis view from `0` to `2π`. Without fixed limits, the axes would auto-rescale as data is added, causing the whole plot to jitter and zoom during the animation. Fixed limits keep the frame stable.

```
ax.set_ylim(-1.2, 1.2)
```

Fixes the y-axis from `-1.2` to `1.2`. The sine ranges from `-1` to `1`, so `±1.2` adds a small margin so the peak doesn't touch the axis border. Again, this keeps the viewport constant.

THE UPDATE FUNCTION

```
def update(frame):
```

Defines the function `FuncAnimation` will call **once per frame**. Matplotlib passes the current frame value as the argument. Here, `frame` will be an integer `0, 1, 2, ..., 199`, because we later pass `frames=len(x)` (an integer), and Matplotlib interprets an integer as `range(0, N)`.

```
    line.set_data(x[:frame], y[:frame])
```

Sets the line's data to the **first `frame` points** of `x` and `y`.

- When `frame == 0`, the slices are empty, so nothing is drawn.
- When `frame == 50`, you see the sine wave up to index 50.
- When `frame == 200`, you see the full wave.

`set_data` is the efficient setter on `Line2D` that replaces the line's data in place. This is what makes the wave appear to "draw itself" progressively.

```
    return line,
```

Returns a **tuple** containing the artist(s) that changed. Two things to note:

1. The **trailing comma** is essential — `return line,` is `return (line,)`, a tuple. `return line` would return the bare object and Matplotlib would complain.
2. This return value is **required when `blit=True`**: Matplotlib needs to know which artists to re-render. Returning the artist from the update function is the standard contract for blitting. If you omit the return (or return `None`), the animation may appear frozen or raise a warning.

CREATING THE ANIMATION

```
ani = animation.FuncAnimation(
```

Constructs the animation object. **You must assign it to a variable.** If you don't keep a reference, Python's garbage collector may delete `ani`, and the animation will stop running (a very common gotcha).

```
    fig, update, frames=len(x), interval=20, blit=True
)
```

- **`fig`** : The figure to animate. `FuncAnimation` needs it to know what to redraw and how to save.
- **`update`** : The callback function described above.
- **`frames=len(x)`** → `frames=200` . An integer tells Matplotlib to call `update` with `0, 1, ..., 199` . It could also be a list/array of arbitrary values, or a generator for infinite/lazy sequences.
- **`interval=20`** : The delay between frames in **milliseconds**. 20 ms ≈ 50 frames per second (theoretical max; actual speed depends on rendering time).
- **`blit=True`** : Enables blitting — Matplotlib caches the static background of the figure and only redraws the artists returned by `update` each frame. This is much faster but requires the update function to return the changed artists (which ours does). If you ever need the full axes to be redrawn every frame (e.g., the title or axis labels change), set `blit=False` .

DISPLAYING THE ANIMATION

```
plt.show()
```

Opens the GUI window and starts the event loop that drives the animation. Without this call (in a script), nothing appears. In a Jupyter notebook, `plt.show()` typically does not animate inline; instead you would use `HTML(ani.to_jshtml())` or `ani.save(...)` to view the result.

Exercises.

1. Draw the graph of $y = e^{-\alpha x^2}$ once, then normalize it. Be careful with the major & minor ticks and compare them next to each other.
2. Display a Galaxy, using `imshow()` method.
3. Display the $3D$ graph of path of an electron in an external magnetic filed.
4. Visualize the motion of a double pendulum with fixed end. Why this phenomenon is called **chaotic**? Could you observe it's chaotic motion in the animation?
5. Animate the motion of an electric dipole in an electric field.